TEAM HF27-0030 • PROBLEM STATEMENT 1 • COMPUTER VISION

SITEGUARD

PPE Compliance Detection — System Architecture and Project Guide

Everything from the first step to today: what the system is, how each part works, which languages, libraries, datasets and models we used, how we trained and tested, what we built, what we archived and what is still planned.

KLS Vishwanathrao Deshpande Institute of Technology, Haliyal • Dept. of CSE & CSE (AI & ML) • Written 30 Oct 2026

1. The project in one page
2. System architecture (diagram)
3. How one photo flows through the system
4. Languages and tools
5. Datasets
6. Models and training
7. The core logic we wrote
8. Module-by-module map
9. Video, accuracy modes, extra PPE
10. Testing and results
11. The journey, step by step
12. Built, archived, planned
13. Folder structure and how to run
14. Limits, licences, credits

1. The project in one page

Problem (from the problem statement): supervisors cannot consistently spot workers without helmets or safety vests across busy industrial areas.

What SiteGuard does: it takes a photo (or a short video), finds every **person**, finds **helmets and vests**, decides which item belongs to which person, applies a rule (helmet and vest required by default), and reports a verdict **per worker**: **COMPLIANT**, **VIOLATION** (with the missing item named) or **NEEDS REVIEW**. It draws the result on the image, produces a table and a violation summary, and exports CSV, JSON and PNG.

> **The central idea.** The problem says a helmet detected anywhere in the image must not clear every worker. So the system never asks "is there a helmet in the picture?". It asks **whose head the helmet is on**. Each worker has a head zone and a torso zone, and each helmet or vest can belong to at most one worker. A helmet on a table or in a hand sits in nobody's head zone and clears nobody.

Problem-statement requirement	How we meet it	Status
Detect people, helmets, safety vests	Fine-tuned YOLO11s (3 classes)	**Built**
Short video clips (optional)	ByteTrack tracking + persistence filter; Video tab in the app	**Built (synthetic test)**
Associate each item with its person	Zone matching, one item to one worker (`associate.py`)	**Built, tested**
Rules per worker, flag the missing item	`rules.py` , configurable required items	**Built**
Annotated images with a per-worker verdict	`annotate.py` : green / red / amber boxes, banner	**Built**
Violation summary	`summary.py` , batch report, PDF notices	**Built**
Multiple workers, occlusion, distance, poor light	Per-worker verdicts, NEEDS REVIEW, zoomed tiles, CLAHE	**Built, partly measured**

2. System architecture

The system is built in layers. Data moves down the page; each layer only talks to the one below it, which is why every part can be tested on its own.



Training pipeline (offline, run once per model)

Public dataset	Prepare script	Prepared set	Training run	Output model
Construction-PPE	`prepare_dataset.py`	`data/ppe3` (person, helmet, vest)	from pretrained YOLO11s, 60 epochs	**v1** `best_v1.pt`
CHVG (VOC XML) + the above	`add_chvg.py`	`data/ppe_merged` (3 classes, 2,491 train)	from v1, 30 epochs	**v2** `best.pt` (default)
Construction-PPE	`prepare_extras.py`	`data/ppe6` (6 classes)	from v2, 40 epochs	**6-class** `extras.pt`

3. How one photo flows through the system

#	Step	File	What happens, with an example
1	Read	`app.py`	The photo is decoded into an image array (OpenCV).
2	Brighten	`enhance.py`	If the average brightness is below 70 (of 255) CLAHE raises local contrast; colours stay true.
3	Detect	`detector.py`	YOLO returns boxes with a label and a confidence, e.g. `helmet 0.62`. Everything above a low "weak" floor (0.15) is kept. Optional zoomed tiles add small background workers.
4	Filter people	`associate.py`	Keep persons above 0.35 confidence whose box is human-shaped (width/height ≤ 2.0).
5	Match	`associate.py`	For each person build a head zone (top 35%, 10% above the box, middle 70% sideways) and a torso zone (20% to 75% down). A helmet is a candidate if its centre is in the head zone and at least half of it overlaps; a vest if its centre is in the torso zone, it is mostly on the person, and big enough. Candidates are scored (60% overlap, 40% closeness to the zone centre) and assigned best first, one item to one worker.
6	Flag	`associate.py`	Warning flags per worker: tiny, edge_top, edge_bottom, torso_cut, low_conf, crowded, wide_box, small_detail, plus weak_helmet / weak_vest when only a faint detection exists.
7	Judge	`rules.py`	Each required item becomes PRESENT, MISSING or UNVERIFIABLE (a flag blocks judging it). Any MISSING → VIOLATION; else any UNVERIFIABLE → NEEDS REVIEW; else COMPLIANT. A reason sentence is written.
8	Draw	`annotate.py`	Green / red / amber person boxes, non-overlapping labels, blue boxes with `vest 0.77` tags, a summary banner. Only people are boxed by default.
9	Summarise	`summary.py`	A per-worker table, counts, a breakdown by missing item, and CSV / JSON / PNG downloads.

> **Why a third outcome?** A worker who is tiny, cut off by the photo edge, hidden in a crowd, or backed only by a faint detection cannot be judged fairly. NEEDS REVIEW sends them to a person instead of making a false accusation. A confident wrong "OK" is worse than either, so most of our fixes were about preventing false passes.

4. Languages and tools

Tool	Version	What we used it for
Python	3.12.10	All of our code: detection wrapper, matching logic, rules, app, tests, training scripts. (The machine also has 3.14; we use a 3.12 virtual environment because PyTorch and Ultralytics support it.)
Ultralytics YOLO11	8.4.193	The detection model family (YOLO11s), training, validation and the built-in ByteTrack tracker
PyTorch + torchvision	2.11.0 / 0.26.0 (CUDA 12.8)	The deep-learning engine under YOLO; runs on the laptop's NVIDIA RTX 5060 GPU
OpenCV	5.0.0.93	Reading images and video, drawing, CLAHE contrast, writing the annotated video
Streamlit	1.66.1	The web app
pandas, NumPy	3.1.0, 2.5.4	Tables, CSV/JSON export, image arrays
SQLite (Python standard library)	built in	The violation register database (no extra server)
ReportLab	5.1.0	PDF violation notices and summary reports
Altair	6.3.0	Charts (installed with Streamlit); used by the archived register UI
lap	0.5.13	Assignment solver required by the ByteTrack tracker
pytest	9.2.0	117 automatic tests
python-pptx	1.0.2	Built the presentation
HTML / CSS	—	Only for the batch report page and for generating these PDFs
PowerShell	5.1	Only for typing run commands on Windows
Claude Code (AI assistant)	—	Used to help write and test the code under our direction; credited in the README

Hardware: a Windows 11 laptop with an NVIDIA GeForce RTX 5060 (8 GB). It needs a CUDA 12.8 build of PyTorch, which is why the install command points at the `cu128` index.

5. Datasets

Dataset	Source and licence	What it is and how we used it
Construction-PPE	Ultralytics AGPL-3.0	1,416 photos (1,132 train / 143 val / 141 test), 11 original classes: helmet, gloves, vest, boots, goggles, none, Person, no_helmet, no_goggle, no_gloves, no_boots. **We kept** person, helmet, vest (`data/ppe3`) and, for the second model, also gloves, boots, goggles (`data/ppe6`). We dropped every `no_*` class on purpose.
CHVG	Md. Ferdous and Sk. Md. Masudul Ahsan, 2022, *PeerJ Computer Science 8:e999* CC BY 4.0	1,699 construction-site photos, 640×640, Pascal VOC XML. Classes: person (4,674 boxes), vest (2,137), helmets in four colours white / yellow / blue / red (3,531 total, mapped to one "helmet" class), head (730, dropped), glass (532, dropped). We converted it to YOLO format and split by source photo: 1,359 train / 167 val / 173 test.
Merged set `data/ppe_merged`	Our own combination	Construction-PPE + CHVG, 3 classes: **2,491 train / 310 val / 314 test**. The original test photos are kept unchanged so old and new models can be compared on the same images.

Our own photos	Supplied by the team	12 real site photos (`eval/cases`), plus 4 earlier test photos. **Test only, never used for training.** Checked by eye (see section 10).
Pretrained start	Ultralytics (YOLO11s weights)	The model already knew everyday objects and people before we touched it (transfer learning).

Box counts in the prepared sets

Set	Split	Images	person	helmet	vest	gloves	boots	goggles
ppe3 / ppe6	train	1,132	1,770	1,341	1,269	1,146*	1,235*	419*
ppe3 / ppe6	val	143	239	201	171	136*	151*	47*
ppe3 / ppe6	test	141	236	192	178	163*	211*	52*

*only in the 6-class set (`ppe6`). Considered but **not used**: SH17 (Kaggle, CC BY-NC-SA 4.0, needs a login), several Roboflow Universe sets, and Hugging Face PPE sets. Several open-source repos were read for **ideas only**; no code was copied.

6. Models and training

Model	Started from	Trained on	Classes	Epochs / time	Final validation mAP@50	File
v1	YOLO11s pretrained	ppe3 (1,132 photos)	person, helmet, vest	60 / 36 min	0.872 (143 photos)	`best_v1.pt`
v2 (default)	v1	ppe_merged (2,491 photos)	person, helmet, vest	30 / 40 min	0.888 (310 photos)	`best.pt`, `best_v2.pt`
6-class (extras)	v2	ppe6 (1,132 photos)	person, helmet, vest, gloves, boots, goggles	40 / about 45 min	0.817 (143 photos)	`extras.pt`

Training settings (all runs): image size 960 px (larger than the 640 default so distant workers stay visible), automatic batch size to fit the 8 GB GPU, stronger brightness variation (`hsv_v=0.5`) for poor lighting, small rotation (5°), mosaic augmentation, early stopping. Command: `python scripts/train.py --data <yaml> --model <start weights> --epochs N --name <run>`.

> **Why training was quick:** transfer learning. The pretrained model already recognised people and objects, so each round only had to adapt to PPE: 36 to 45 minutes on a laptop GPU instead of days.

Why two rounds?

Round 1 scored well on photos like its own training photos (0.866) but only 0.758 on photos from other sites. Adding a second dataset with different sites and four helmet colours (round 2) was the answer. Round 2 is slightly lower on the original split (0.829) but found more workers and fixed two wrong verdicts on the team's own photos, so it became the default.

Held-out photos (mAP@50)	Round 1	Round 2	Note
Original test split (141)	0.866	0.829	Same source as the training photos
CHVG test split (173)	0.758	0.882	Round 2 trained on the training half of CHVG, so this is **not** a like-for-like comparison

7. The core logic we wrote

7.1 Zones (fractions of the person box height, from its top)

Item	Zone	Extra conditions
Helmet	from 10% above the box to 35% down; sideways the middle 70%	Centre inside the zone, at least 50% of the helmet overlaps it. The sideways limit stops an oversized box from taking a neighbour's helmet
Vest	20% to 75% down (75% so seated and bending workers still match)	At least 60% of the vest on the person, width at least 45% and height at least 20% of the person's
Goggles	5% above to 28% down, slightly wider than the person	Small item: the worker must be at least 120 px tall (or 18% of the image)
Gloves	25% to 85% down, 30% wider each side (hands swing out)	A worker may own several; at least 120 px tall
Boots	78% to 112% down (feet sit at and below the box edge), 25% wider	A worker may own several

7.2 The decision rules

- **One item, one worker:** candidates are ranked by score and assigned best first. If two workers score within 0.02 of each other, the item goes to the one with the smaller box (the larger box is more likely an oversized detection).
- **Weak evidence never accuses:** a helmet or vest seen between the weak floor (0.15) and its threshold (0.30 helmet, 0.35 vest) stops the worker being called a violation. They go to NEEDS REVIEW.
- **Cut-off logic:** head cut off blocks the helmet; legs cut off blocks boots; the vest is blocked only if the box is short for its width (torso really cut), so a worker cut off at the knees can still be judged.
- **Missing is inferred:** we do not train a "no helmet" class. A missing item is the absence of a match.

7.3 Extra techniques

Technique	What it does and the trade-off
Zoomed tiles	The photo is also cut into four overlapping pieces, each checked at full resolution, so small background workers are found. Boxes sliced by a tile edge are discarded, and a person seen only in a tile needs 0.55 confidence. Trade-off: a few extra or duplicate boxes, slower.
Duplicate merging	Same-class boxes that overlap heavily, or that lie almost wholly inside a more confident box, are merged.
Test-time augmentation	The model looks at each photo flipped and rescaled and averages, steadying borderline scores.
Ensemble fusion	The 6-class model gives a second opinion on person / helmet / vest. Boxes both agree on get the average confidence; a lone box keeps half. **Experimental:** on our photos both models read a knitted beanie as a helmet, so agreement made that mistake worse. Not the default.
CLAHE	Contrast boost for dark photos, on the lightness channel so a vest's orange stays orange.
Video persistence	Each person keeps a track ID (ByteTrack). A violation is confirmed only when seen in 5 of the last 8 *usable* frames; NEEDS REVIEW frames are skipped; each confirmed violation is reported once and saves an evidence photo and a CSV row.

8. Module-by-module map

File	Lines	Responsibility
`ppe/types.py`	54	Plain data classes passed between modules: Detection, WorkerMatch, Verdict, AnalysisResult; the item list and status names

`ppe/geometry.py`	33	Box maths: area, overlap (IoU), "how much of A is inside B", clipping
`ppe/config.py`	68	Every tunable number in one place: thresholds, zones, flags, tile settings, video persistence
`ppe/detector.py`	201	Wraps the YOLO models; class names mapped by name; tiling, merging, fusion, tracking, extras
`ppe/enhance.py`	17	Dark-photo detection and CLAHE
`ppe/associate.py`	181	**Core.** Zones, scoring, one-to-one assignment, weak evidence, flags
`ppe/rules.py`	86	**Core.** Item states, verdicts, reason sentences
`ppe/pipeline.py`	41	Glue: enhance → detect → associate → rules; also the single-image command line
`ppe/annotate.py`	89	Boxes, readable labels in crowds, item tags, summary banner, people-only mode
`ppe/summary.py`	56	Per-worker table (columns follow the required items), counts, CSV, JSON
`ppe/batch.py`	86	Analyse a whole folder (optionally watch it) and write an HTML report
`ppe/track.py`	103	Video: persistence filter, video processing, evidence frames, incidents CSV
`ppe/store.py`	207	SQLite register: workers, violations, event log (time log), checks (compliance rate)
`ppe/evidence.py`, `service.py`	20, 81	Save evidence photos; turn a checked photo into register records
`ppe/report.py`	127	PDF violation notice and summary, with evidence image, time log and worker name
`ppe/insights.py`, `demo.py`	103, 47	Safety score, streaks, patterns, tips; demo data for presentations
`app.py`	~190	Streamlit app: Photos tab, Video tab, settings (layout only)
`scripts/`	—	`prepare_dataset.py`, `add_chvg.py`, `prepare_extras.py`, `train.py`
`eval/run_eval.py`	—	Scores any photo set against a hand-written answer key, by group
`tests/`	10 files	117 automatic tests with hand-made boxes (no model needed) plus fake detectors and synthetic video
`archive/register_ui/`	—	The three-page Supervisor / Employee / Register interface, set aside on request

9. Video, accuracy modes and extra PPE

Feature	Status and detail
Video upload	**Built. Video tab: upload, progress bar, confirmed violations table, evidence photos, annotated video, CSV. Tested end to end on a synthetic clip made from a real photo (6 violations confirmed). Not yet tested on real footage; speed is about 0.5 s per analysed frame on this laptop.**
Accuracy modes (photos)	**Fast**: one pass. **Balanced** (default): plus zoomed tiles. **Ensemble (experimental)**: two models and augmentation; steadier vest scores but a risk of accepting caps as helmets.
Gloves, boots, goggles	**Matching zones, rules and tests are built and the 6-class model is trained (val mAP@50 0.817). The app offers these items when `weights/extras.pt` exists. Not yet scored per item on a labelled test set, so no accuracy is claimed.**
People-only drawing	Only people are boxed by default. Helmets or vests nobody is wearing are hidden unless the sidebar option is on. Boxes far wider than tall are ignored as non-human.

10. Testing and results

Automatic tests: 117 pass

They cover matching (helmet in hand, on a table, between two workers, an oversized box, seated posture), rules and verdicts, extra items, weak evidence, tiling and fusion, video persistence (including a full run on a synthetic clip), the register and PDFs, insights, drawing, and the evaluation logic.

Detection quality on 314 held-out photos (AP@0.5, full pipeline)

Configuration	Person	Helmet	Vest	Speed per photo
One model, tiles on (default before changes)	0.868	0.924	0.864	0.16 s
Ensemble + augmentation, tiles off	0.891	0.910	0.882	0.18 s
Ensemble + augmentation, tiles on	0.847	0.920	0.892	0.68 s

Real-photo review by eye (12 photos from the team)

Method: run the system, compare each result with the original photo, re-inspect every disagreement, fix logic faults, re-run. Of **45 workers a person could clearly judge: 34 correct (76%), 9 sent to review where the answer was clear (20%), 2 wrong (4%).** Among decisive verdicts 34 of 36 were right. Truth was written by one reviewer, so treat this as indicative, not a formal benchmark.

Hard case from the problem	Result
Helmet on a table	**Not credited to anyone; bare-headed seated workers flagged or sent to review**
Helmet held in hand	**Not credited; both workers flagged NO HELMET**
Cap, beanie, head cloth instead of a helmet	**Mostly right; one baseball cap was accepted as a helmet**
Hi-vis shirt or life jacket that is not a vest	**Not solved: the model counts them as vests**
Small, background and hidden workers	**Better with tiles; about 8 to 10 clearly visible workers still missed**

Faults found by checking, and fixed

- **False pass:** an oversized person box credited a neighbour's helmet at its edge → helmets must now sit in the middle of the worker's box.
- **Seated workers' vests not matched** → torso zone widened to 75%.
- **Vest "cannot verify" when only the legs were cut off** → new `torso_cut` rule.
- **A borderline helmet caused a false violation** → weak-evidence rule.
- **One helmet detected twice appeared as "unassigned"** → duplicate merging.

11. The journey, step by step

#	Step	What we did
1	Understand the problem	Read the problem statement; identified the constraint (a helmet anywhere must not clear everyone) and the hard cases.
2	Research	Studied open-source PPE projects and public datasets for ideas only. Found none matched each item to the individual worker, so we built that.
3	Environment	Python 3.12 virtual environment, CUDA 12.8 PyTorch, Ultralytics, OpenCV, Streamlit; confirmed the GPU is used.

4	Data round 1	Downloaded Construction-PPE; reduced it to person, helmet, vest.
5	Train v1	Fine-tuned YOLO11s: 60 epochs, 36 minutes.
6	Core logic and tests	Wrote and tested association, rules, summary and drawing before relying on the model.
7	App, command line, batch tool	Photo upload, annotated output, table, downloads; folder analyser with HTML report.
8	Check on real photos	Found and fixed: false violations on borderline helmets (weak-evidence rule), duplicate boxes, crowd labels, and added zoomed tiles for small workers.
9	Video	ByteTrack tracking and a persistence filter so a single bad frame does not raise an alarm.
10	Data round 2 and train v2	Added CHVG (converted from XML, split by source photo to avoid leakage); fine-tuned v1 for 30 epochs on the merged set.
11	Violation register	SQLite register with workers, violations, a time log, PDF notices, safety scores and analytics; a three-page Supervisor / Employee / Register interface. Later set aside to return to the simple app.
12	Extra PPE	Added gloves, boots and goggles zones, rules and tests; trained a 6-class model.
13	Evaluate honestly	Held-out scoring on 314 photos; 12-photo review by eye, three passes; fixed the false pass, seated vests and torso-cut logic.
14	Accuracy modes	Tested tiles, augmentation and ensembling; kept Balanced as default because the ensemble made a false pass worse.
15	Clearer output	Summary banner, item tags, people-only drawing, non-human box filter.
16	Video upload in the app	Video tab with progress, evidence and annotated video.
17	Documents	README, module walkthrough, demo script, team guide, judge Q&A, results review, presentation, and this guide.

12. Built, archived and planned

Part	Status	Notes
Detection, matching, rules, verdicts, annotated output, summary, exports	**Built, tested**	The core of the project
Web app with Photos and Video tabs, batch tool, command line	**Built**	Video verified on a synthetic clip only
Violation register, PDF notices, safety scores, demo data	**Built, UI archived**	Modules and tests remain in `ppe/`; the three pages are in `archive/register_ui/`
Gloves, boots, goggles	**Logic + model built**	Per-item accuracy not yet measured
Job roles with editable outfit rules in a config file	**Planned**	Standards must come from the client; none invented in code
Worker profiles; names confirmed by a supervisor before any violation is saved under them	**Planned**	No automatic naming. Face matching, if ever used, optional, off by default, documented
Supervisor notifications, dispute handling	**Planned**	In-app first; email optional
Repeat-violation escalation including supervisors, with human review and an audit log	**Planned**	No formal notice without a human review step

Users with roles and access control; consent, retention, hashed passwords; privacy note (DPDP Act 2023 guidance)	**Planned**	Required before any real deployment

13. Folder structure and how to run

```
D:\software\ppe-detection\
  app.py                      Streamlit app (Photos + Video)
  ppe\                        the system (types, geometry, config, detector, enhance, associate, rules,
                              pipeline, annotate, summary, batch, track, store, evidence, report,
                              service, insights, demo)
  scripts\                    prepare_dataset.py, add_chvg.py, prepare_extras.py, train.py
  tests\                      10 test files, 117 tests
  eval\                       run_eval.py, ground_truth.csv, test_images\, cases\ (12 real photos)
  weights\                    best.pt (v2, default), best_v1.pt, best_v2.pt, extras.pt (6-class)
  data\                       raw\, ppe3\, ppe6\, ppe_merged\, extra\chvg\   (not committed)
  runs\                       training runs and validation output
  outputs\                    annotated results, batch reports, video runs, evidence
  docs\                       README-style guides, CASE_REVIEW.md, WALKTHROUGH.md, DEMO.md, PDFs, the PPTX
  archive\register_ui\        the three-page register interface (set aside)
  .streamlit\config.toml      theme and upload limit
  requirements.txt, README.md
```

Run it (Windows PowerShell)

```
py -3.12 -m venv .venv ; .\.venv\Scripts\activate
python -m pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128
python -m pip install -r requirements.txt
streamlit run app.py                                          # the web app
python -m ppe.pipeline --image photo.jpg --save out.png       # one image from the command line
python -m ppe.batch --folder eval\test_images                 # a folder -> HTML report
python eval\run_eval.py --weights weights\best.pt             # score against ground_truth.csv
python -m pytest                                              # 117 tests
python scripts\train.py --data data\ppe_merged\data.yaml --model weights\best_v1.pt --epochs 30 --name ppe_v2
```

The numbers that matter most (all in `config.py`)

Setting	Value	Setting	Value
Person confidence	0.35	Helmet confidence	0.30
Vest confidence	0.35	Weak-evidence floor	0.15
Head zone	top 35% (+10% above)	Torso zone	20% to 75%
Dark-photo threshold	mean brightness < 70	Image size	960 px
Tiles	2×2, 25% overlap	Video confirmation	5 of the last 8 usable frames

14. Limits, licences and credits

What the system does not do (be upfront about these)

- It cannot tell a **worker** from a passer-by or a person in an area where PPE is not required; it judges every person it sees.
- A knitted cap or beanie can be read as a helmet; hi-vis shirts and life jackets can pass as vests. This needs training photos of those items, not more models.
- Small, background and hidden workers can be missed. Seated or bending workers often go to NEEDS REVIEW.

- It does **not** recognise faces or identify people. Names would only ever be attached by a supervisor.
- Video has not yet been tested on real footage; real-time speed is not benchmarked.
- Accuracy for the hard cases is from a 12-photo review, not a formal benchmark; scores on the original split flatter the model because the data has many near-identical frames.

Licences and credits

Component	Licence / credit
Ultralytics YOLO11 and the Construction-PPE dataset	AGPL-3.0, so our source is published with the project
CHVG dataset	CC BY 4.0: Md. Ferdous and Sk. Md. Masudul Ahsan, PeerJ Computer Science 8:e999 (2022), doi 10.7717/peerj-cs.999
OpenCV, Streamlit, pandas, NumPy, PyTorch, ReportLab, pytest	Open-source libraries under their own licences
Weights trained on the datasets	Inherit the datasets' terms
Ideas studied	Several open-source PPE projects, concepts only (worker tracking, violation CSVs, flicker and shared-equipment weaknesses). No code copied.
AI assistance	Claude (Anthropic) helped write and test the code under the team's direction; the team reviews the work

> **One-sentence summary for judges:** "We fine-tuned a pretrained YOLO model on two public datasets for detection, and wrote the matching, rules, summary, app and tests ourselves, so every helmet and vest is tied to the specific worker wearing it, and a worker we cannot judge fairly is sent for review instead of being accused."